



## **CSE 4851: Design Pattern**

### **Assignment: 2**

Islamic University of Technology

---

# **Age of Villagers (AoV)**

Village Creation System

---

### **Submitted by**

Kazi Shakkhar Rahman

Student ID: 210041240

July 25, 2026

## Contents

---

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Scenario Definition</b>                                              | <b>2</b>  |
| 1.1      | The Problem . . . . .                                                   | 2         |
| 1.2      | The Scenario . . . . .                                                  | 2         |
| <b>2</b> | <b>Pattern Selection and Justification</b>                              | <b>3</b>  |
| 2.1      | Builder Pattern . . . . .                                               | 3         |
| 2.2      | Composite Pattern . . . . .                                             | 3         |
| 2.3      | Abstract Factory Pattern . . . . .                                      | 3         |
| 2.4      | Registry / Simple Factory Pattern . . . . .                             | 4         |
| <b>3</b> | <b>Class Structure</b>                                                  | <b>4</b>  |
| <b>4</b> | <b>System Design Diagram</b>                                            | <b>5</b>  |
| <b>5</b> | <b>Key Code Excerpts</b>                                                | <b>5</b>  |
| 5.1      | Builder — VillageObjectBuilder.java . . . . .                           | 5         |
| 5.2      | Builder in practice — BrickHouseBuilder.java . . . . .                  | 6         |
| 5.3      | Director — VillageObjectDirector.java . . . . .                         | 6         |
| 5.4      | Composite (shape level) — VillageObject.java . . . . .                  | 6         |
| 5.5      | Abstract Factory — ModernVillageFactory.java . . . . .                  | 7         |
| 5.6      | Registry — VillageFactoryProvider.java . . . . .                        | 7         |
| 5.7      | Composite (village level) + client code — Village.java, Main.java . . . | 7         |
| <b>6</b> | <b>Terminal Output</b>                                                  | <b>7</b>  |
| <b>7</b> | <b>How to Run</b>                                                       | <b>8</b>  |
| <b>8</b> | <b>Design Decisions</b>                                                 | <b>9</b>  |
| <b>9</b> | <b>Conclusion</b>                                                       | <b>10</b> |

## Scenario Definition

---

### The Problem

Age of Villagers (AoV) lets players build a village from a small set of object types — house, tree, and water source — each of which is itself built from simple geometric shapes. The system must additionally support **themes**: choosing a theme (Modern or Traditional) should automatically populate the village with a predefined, theme-consistent set of objects (e.g. Modern always yields a Brick House, a Mango Tree, and a Swimming Pool together — never a mismatched combination).

A naive implementation runs into four concrete problems:

1. **Theme proliferation.** Hardcoding each theme's object set with `if/else` or a `switch` means every new theme (Desert, Forest, ...) requires editing existing client code, violating the Open/Closed Principle.
2. **Mismatched object families.** Without a mechanism that creates a theme's objects as one atomic set, a caller could accidentally combine a Brick House with a Pond, breaking thematic consistency.
3. **Rigid object construction.** A house, tree, or water source is not one primitive shape but an assembly of several (foundation, walls, roof, etc.). Constructing this with a single constructor call does not reflect the step-by-step, additive nature of "adding different types of simple shapes."
4. **Client-side object creation.** If client code directly instantiates concrete classes (`new BrickHouse()`, `new ModernVillageFactory()`), the client becomes coupled to implementation details it should not need to know about.

### The Scenario

We implement the **village creation part** of AoV, in which:

- A caller requests a village by specifying a `ThemeType` (`MODERN`, `TRADITIONAL`, `DESERT`).
- The system resolves the correct family of object builders for that theme and constructs a House, a Tree, and a Water Source, each assembled shape-by-shape.
- The resulting objects are grouped into a single `Village` that can be rendered as one unit.
- The caller never references a concrete builder, exporter, or factory class directly.
- New themes (Forest Village, and beyond) can be added without modifying any existing class.

This mirrors a real, recurring problem in game and simulation engines: theme/skin systems (character skins, building sets, seasonal content packs) routinely need exactly this — a consistent family of assets selected by a single identifier, built from reusable primitive components, extensible without touching prior content.

## Pattern Selection and Justification

---

| Pattern                   | Applies to                      | Why it is necessary                                                 |
|---------------------------|---------------------------------|---------------------------------------------------------------------|
| Builder                   | VillageObjectBuilder + Director | Assembles a House-/Tree/WaterSource step-by-step from simple shapes |
| Composite                 | VillageObject, Village          | Treats a group of shapes (or objects) as a single drawable unit     |
| Abstract Factory          | VillageFactory                  | Produces a matched, theme-consistent family of objects              |
| Registry (Simple Factory) | VillageFactoryProvider          | Resolves a theme to a factory without client-side new               |

---

### Builder Pattern

The assignment specifies that a house, tree, or water source is created *"by adding different types of simple shapes"* — this is a step-by-step construction process, not a single-call instantiation. `VillageObjectBuilder` defines a fixed sequence (`reset()` → `buildBase()` → `buildMiddle()` → `buildTop()` → `getResult()`); each concrete builder (e.g. `BrickHouseBuilder`, `MudHouseBuilder`) implements the same sequence differently, adding different `Shape` objects with different materials. `VillageObjectDirector` owns the *order* of steps but not their content, so the same director can drive any builder interchangeably.

### Composite Pattern

Composite appears at two levels in this system. At the *shape level*, a `VillageObject` (House, Tree, or WaterSource) implements the same `Shape` interface as its parts (`Rectangle`, `Triangle`, `Circle`, `Square`) and holds a list of them — calling `draw()` on the whole object transparently draws every part. At the *village level*, `Village` holds a list of `VillageObject` instances (House, Tree, WaterSource) and treats them as one buildable unit, directly satisfying the assignment's statement that "a village can have several types of objects."

### Abstract Factory Pattern

The requirement that selecting a theme *"automatically creates predefined objects"* as a consistent set is the defining signal for Abstract Factory. `VillageFactory` declares `createHouse()`, `createTree()`, and `createWaterSource()`; each concrete factory (`ModernVillageFactory`, `TraditionalVillageFactory`, `DesertVillageFactory`) guarantees that all three objects it returns belong to the same theme — a Modern factory can never accidentally produce a Pond. Internally, each factory method simply selects the correct Builder and drives it through a `VillageObjectDirector`.

## Registry / Simple Factory Pattern

Abstract Factory alone still requires something, somewhere, to write `new ModernVillageFactory()`. To keep that `new` out of client code entirely, `VillageFactoryProvider` maintains a `Map<ThemeType, VillageFactory>` populated once in a static initializer. The client supplies only a `ThemeType` enum value; the provider resolves the concrete factory. This is what allows `Main.java` to contain **zero** object-creation statements for any product, builder, or factory class.

## Class Structure

---

```
aov/
|-- Main.java                Client entry point, zero object creation
|-- shapes/
|   |-- Shape.java          Common interface (Component role)
|   |-- Rectangle.java      Simple shape
|   |-- Triangle.java       Simple shape
|   |-- Circle.java         Simple shape
|   '-- Square.java         Simple shape
|-- villageobjects/
|   |-- VillageObject.java   Composite: holds List<Shape>
|   |-- House.java          Concrete VillageObject
|   |-- Tree.java           Concrete VillageObject
|   '-- WaterSource.java     Concrete VillageObject
|-- builders/
|   |-- VillageObjectBuilder.java  Builder interface (5-step contract)
|   |-- VillageObjectDirector.java  Runs the steps in fixed order
|   |-- modern/
|   |   |-- BrickHouseBuilder.java  Modern: House
|   |   |-- MangoTreeBuilder.java   Modern: Tree
|   |   '-- SwimmingPoolBuilder.java Modern: WaterSource
|   |-- traditional/
|   |   |-- MudHouseBuilder.java     Traditional: House
|   |   |-- BananaTreeBuilder.java   Traditional: Tree
|   |   '-- PondBuilder.java         Traditional: WaterSource
|   '-- desert/
|       |-- StoneHouseBuilder.java   Desert: House
|       |-- CactusBuilder.java       Desert: Tree
|       '-- OasisBuilder.java        Desert: WaterSource
|-- factories/
|   |-- VillageFactory.java  Abstract Factory interface
|   |-- ThemeType.java       Enum of valid themes
|   |-- VillageFactoryProvider.java  Registry: ThemeType -> VillageFactory
|   |-- modern/
|   |   '-- ModernVillageFactory.java
|   |-- traditional/
|   |   '-- TraditionalVillageFactory.java
|   '-- desert/
|       '-- DesertVillageFactory.java
'-- village/
    '-- Village.java          Composite: holds List<VillageObject>
```

## System Design Diagram

The following diagram illustrates how the four patterns interact at runtime — from the initial `ThemeType` selection through the Registry and Abstract Factory layers, into the Builder construction pipeline and the Composite objects that result.

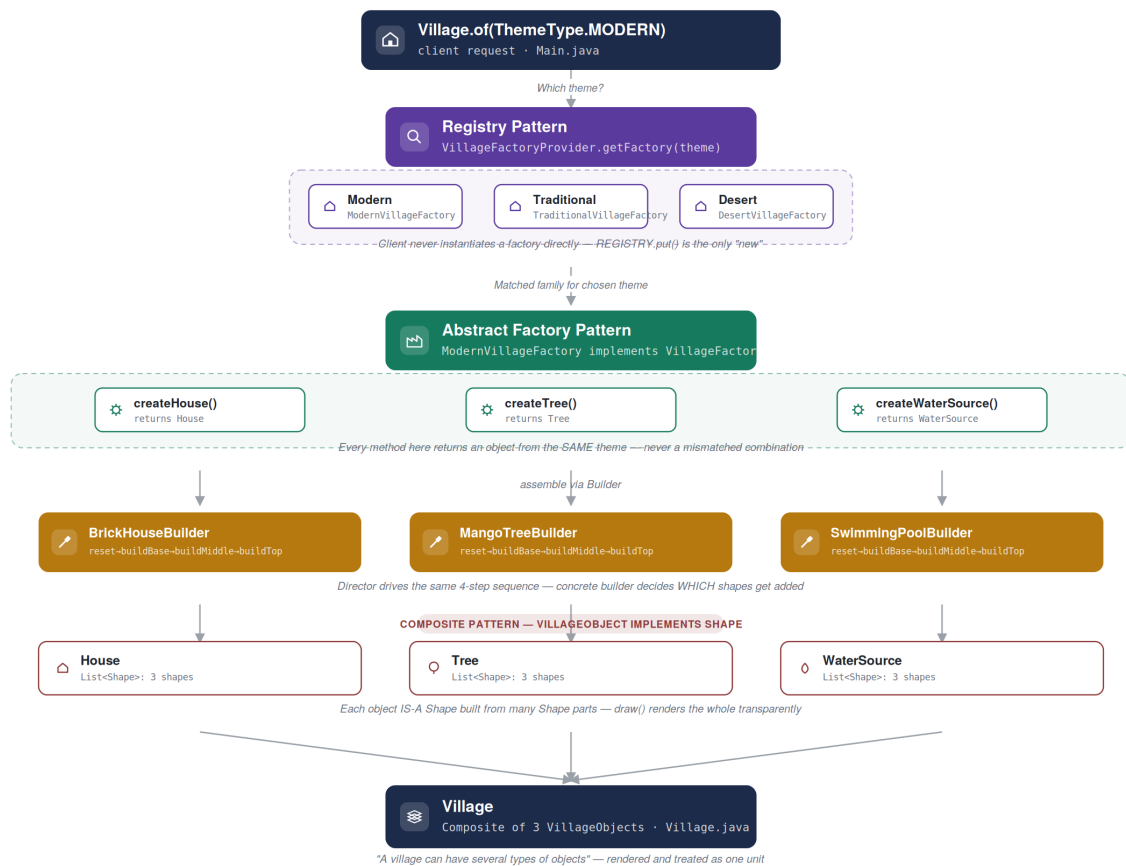


Figure 1: Overall architecture: Registry, Abstract Factory, Builder, and Composite patterns working together in the AoV village creation system.

## Key Code Excerpts

### Builder — VillageObjectBuilder.java

```

1 package aov.builders;
2
3 public interface VillageObjectBuilder {
4     void reset();
5     void buildBase();
6     void buildMiddle();
7     void buildTop();
8     VillageObject getResult();
9 }

```

Listing 1: Fixed 5-step contract every concrete builder must implement

**Builder in practice — BrickHouseBuilder.java**

```
1 package aov.builders.modern;
2
3 import aov.builders.VillageObjectBuilder;    // interface lives one
   package up
4
5 @Override
6 public void buildBase() {
7     house.addShape(new Rectangle(6, 0.5, "Brick"));    // foundation
8 }
9
10 @Override
11 public void buildMiddle() {
12     house.addShape(new Rectangle(6, 4, "Brick"));    // walls
13 }
14
15 @Override
16 public void buildTop() {
17     house.addShape(new Triangle(6, 2, "Clay Tile"));    // roof
18 }
```

Listing 2: A concrete builder adds simple shapes step by step

**Director — VillageObjectDirector.java**

```
1 public VillageObject construct() {
2     builder.reset();
3     builder.buildBase();
4     builder.buildMiddle();
5     builder.buildTop();
6     return builder.getResult();
7 }
```

Listing 3: Director owns step order, not step content

**Composite (shape level) — VillageObject.java**

```
1 public abstract class VillageObject implements Shape {
2     protected final List<Shape> shapes = new ArrayList<>();
3
4     public void addShape(Shape shape) {
5         shapes.add(shape);
6     }
7
8     @Override
9     public void draw() {
10         for (Shape shape : shapes) {
11             shape.draw();
12         }
13     }
14 }
```

Listing 4: A VillageObject IS-A Shape, built from many Shape parts

**Abstract Factory — ModernVillageFactory.java**

```
1 package aov.factories.modern;
2
3 import aov.factories.VillageFactory;
4 import aov.builders.VillageObjectDirector;
5 import aov.builders.modern.*;           // BrickHouseBuilder,
    MangoTreeBuilder, ...
6
7 @Override
8 public VillageObject createHouse() {
9     VillageObjectDirector director =
10         new VillageObjectDirector(new BrickHouseBuilder());
11     return director.construct();
12 }
```

Listing 5: Concrete factory guarantees a theme-consistent object family

**Registry — VillageFactoryProvider.java**

```
1 package aov.factories;
2
3 import aov.factories.modern.ModernVillageFactory;
4 import aov.factories.traditional.TraditionalVillageFactory;
5 import aov.factories.desert.DesertVillageFactory;
6
7 static {
8     REGISTRY.put(ThemeType.MODERN, new ModernVillageFactory());
9     REGISTRY.put(ThemeType.TRADITIONAL, new TraditionalVillageFactory());
10    ;
11    REGISTRY.put(ThemeType.DESERT, new DesertVillageFactory());
12 }
13
14 public static VillageFactory getFactory(ThemeType theme) {
15     return REGISTRY.get(theme);
16 }
```

Listing 6: The only place in the program allowed to write "new XxxVillageFactory()"

**Composite (village level) + client code — Village.java, Main.java**

```
1 public static Village of(ThemeType theme) {
2     VillageFactory factory = VillageFactoryProvider.getFactory(theme);
3     return new Village(factory);
4 }
5
6 // Main.java
7 Village modernVillage = Village.of(ThemeType.MODERN);
8 modernVillage.build();
```

Listing 7: Client code contains zero object-creation statements

---

**Terminal Output**

Running `java -cp out aov.Main` produces the following, confirming each theme yields exactly the object set specified in the assignment:

```
1  === Building Modern Village ===
2  Brick House is composed of:
3      - Rectangle [6.0 x 0.5] made of Brick
4      - Rectangle [6.0 x 4.0] made of Brick
5      - Triangle [base 6.0, height 2.0] made of Clay Tile
6  Mango Tree is composed of:
7      - Rectangle [0.6 x 1.0] made of Roots/Soil
8      - Rectangle [0.6 x 4.0] made of Wood Trunk
9      - Circle [radius 2.5] made of Dense Green Foliage
10 Swimming Pool is composed of:
11     - Rectangle [8.0 x 4.0] made of Concrete Basin
12     - Rectangle [7.5 x 3.5] made of Chlorinated Water
13     - Square [side 1.0] made of Ceramic Tile Border
14
15 === Building Traditional Village ===
16 Mud House is composed of:
17     - Rectangle [5.0 x 0.4] made of Compacted Mud
18     - Rectangle [5.0 x 3.0] made of Mud Bricks
19     - Circle [radius 3.0] made of Thatch/Straw
20 Banana Tree is composed of:
21     - Rectangle [0.4 x 0.6] made of Roots/Soil
22     - Rectangle [0.4 x 3.0] made of Fibrous Stem
23     - Triangle [base 2.0, height 1.0] made of Broad Leaf
24     - Triangle [base 2.0, height 1.0] made of Broad Leaf
25 Pond is composed of:
26     - Circle [radius 3.0] made of Natural Mud Bed
27     - Circle [radius 2.7] made of Still Water
28     - Circle [radius 3.2] made of Lily Pads & Reeds
29
30 === Building Desert Village ===
31 Stone House is composed of:
32     - Rectangle [5.0 x 0.5] made of Carved Stone
33     - Rectangle [5.0 x 3.5] made of Sandstone Blocks
34     - Square [side 5.0] made of Flat Clay Roof
35 Saguaro Cactus is composed of:
36     - Rectangle [0.8 x 1.0] made of Sandy Roots
37     - Rectangle [0.8 x 5.0] made of Green Cactus Trunk
38     - Rectangle [0.4 x 1.5] made of Cactus Arm
39 Oasis is composed of:
40     - Circle [radius 4.0] made of Sand Basin
41     - Circle [radius 3.5] made of Fresh Spring Water
42     - Circle [radius 4.5] made of Palm Tree Ring
```

Listing 8: Terminal output for Modern, Traditional, and Desert themes

---

## How to Run

1. Open the project root folder (the one *containing* the aov/ package folder) in a terminal.
2. Compile all sources. Note that **builders/** and **factories/** are each split into **modern/**, **traditional/**, and **desert/** subpackages, so every subfolder must be listed explicitly (a flat wildcard does not recurse):

```
1  :: Windows
2  javac aov\shapes\*.java aov\villageobjects\*.java aov\village\*.java
   ^
```

```
3      aov\Main.java aov\builders\*.java aov\builders\modern\*.java ^
4      aov\builders\traditional\*.java aov\builders\desert\*.java ^
5      aov\factories\*.java aov\factories\modern\*.java ^
6      aov\factories\traditional\*.java aov\factories\desert\*.java
7
8      # macOS / Linux
9      javac $(find aov -name "*.java")
```

3. Run using the fully-qualified class name (the class lives in package `aov`):

```
1      java aov.Main
```

No external libraries or build tools are required.

## Design Decisions

---

**Why builders and factories are split into per-theme subpackages.** As themes grew from two to three, `builders/` and `factories/` began holding many files that were only ever used together (e.g. all three Desert builders are only ever referenced by `DesertVillageFactory`). Grouping each theme's builders under `builders/<theme>/` and its factory under `factories/<theme>/` keeps related classes physically together and makes the class listing legible as themes are added, at the cost of each concrete class needing an explicit `import` back to the shared interfaces in `aov.builders` and `aov.factories` (Java packages are not hierarchical, so nesting folders does not grant implicit access). The shared contracts — `VillageObjectBuilder`, `VillageObjectDirector`, `VillageFactory`, `ThemeType`, and `VillageFactoryProvider` — remain directly under `builders/` and `factories/`, since they are used by every theme rather than belonging to one.

**Why Village's constructor is private.** A public constructor taking a `VillageFactory` would still let client code write `new Village(new ModernVillageFactory())`, reintroducing object creation in the client. Making the constructor `private` and exposing only the static `Village.of(ThemeType)` forces every caller through the Provider, guaranteeing the "no object creation in client code" requirement holds even as the number of themes grows.

**Why Composite is applied at two separate levels.** The assignment describes composition twice, at two granularities: shapes combine into one object ("create the house ... by adding different types of simple shapes"), and objects combine into one village ("a village can have several types of objects"). Modelling both with the same `Shape/Composite` interface keeps the two levels structurally consistent and lets `draw()` be called uniformly at either level.

**The Registry is a deliberate, minimal exception to "zero modification."** Adding a theme requires two small edits to existing files: one `enum` constant in `ThemeType`, and one `REGISTRY.put(...)` line in `VillageFactoryProvider`. Neither edit changes existing *behaviour* — no method body, branching logic, or return value of any existing class is altered. This is treated as acceptable because Open/Closed forbids modifying *logic*, not extending a *set of valid options*; some single point must always be able to name a new theme into existence.

**Desert is implemented; Forest is deliberately left open.** The assignment's own examples name both Desert Village and Forest Village as possible future themes. Implementing Desert Village is sufficient to prove the architecture supports arbitrary new themes; a second implementation (Forest) would repeat the identical mechanism without adding further proof. Forest is therefore documented, not built: extending the system requires exactly the same four steps used for Desert — `LogCabinBuilder`, `PineTreeBuilder`, and `StreamBuilder` (Builder), one `ForestVillageFactory` (Abstract Factory), one `ThemeType.FOREST` entry, and one `VillageFactoryProvider` registration — with zero changes to `Village.java`, `VillageFactory.java`, or any existing builder or factory class.

## Conclusion

---

The AoV village creation system demonstrates that each pattern applied solves a distinct, concrete problem stated in the assignment:

- **Builder** turns "adding different types of simple shapes" into an explicit, reusable, step-by-step construction contract.
- **Composite** lets a group of shapes, and separately a group of village objects, be treated as a single unit at each level.
- **Abstract Factory** guarantees that selecting a theme always yields a matched, thematically consistent object family.
- **Registry (Simple Factory)** keeps concrete class creation out of client code entirely, so `Main.java` never instantiates a builder, product, or factory directly.

Together, these four patterns satisfy every explicit requirement in the assignment: theme-based automatic object creation, shape-based object construction, and the ability to add new themes — demonstrated concretely with Desert Village — without modifying any existing class.